

Reviewer Pack — Minimal Rank of Braided Tensor Categories over Communication...

Entry f36368cc8b51 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 21:26:22 UTC

Minimal Rank of Braided Tensor Categories over Communication Complexity for Graph Isomorphism

Entry ID: f36368cc8b51 Verdict: INCONCLUSIVE Recorded: 2026-05-25 21:26:22 UTC

1. Conjecture

Field A (mathematical branch): Higher Category Theory (Braided Tensor Categories) **Field B** (complexity object): Complexity Theory: Communication Complexity for Graph Isomorphism

Statement:

{‘quantitative_relation’: ‘ $E[X(\text{instance})] = \Theta(2^{\tau(G)})$ ’, ‘property_relation’: ‘For all instances of graph isomorphism problem, the minimal rank $\tau(G)$ of the associated braided tensor category is lower-bounded by $2^{\Omega(|G|)}$ ’, ‘falsifier_friendly’: ‘If there exists a graph G with $|G| \leq 40$ vertices for which the minimal rank $\tau(G) = O(1)$, then the conjecture is refuted.’}

Rationale (proposer’s reasoning):

{‘explanation’: ‘Braided tensor categories can encode non-trivial algebraic structures that are relevant to complex computations, including those in communication complexity. A high minimal rank could imply a need for exponential communication, suggesting that certain graph isomorphism instances are hard.’, ‘potential_structure’: ‘The conjecture bridges the gap between algebraic structures and complexity measures by exploring the role of braided tensor categories in encoding graph properties.’}

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d8bdd788cab3f9f0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A conjecture is supported if, across all graphs G with $|G| \leq 40$ vertices, the computed minimal rank $\tau(G)$ of the braided tensor category associated with G exhibits an exponential growth rate, as evidenced by at least 80% (support_fraction ≥ 0.8) of the seeds showing a correlation coefficient greater than or equal to 0.9 between $\tau(G)$ and $2^{\tau(G)}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "braided tensor categories" AND "communication complexity" AND graph isomorphism" - "minimal rank" AND braided tensor categories AND communication complexity" - "graph isomorphism problem" AND lower bound" AND minimal rank braided tensor category"

Top relevant hits considered: - [http://arxiv.org/abs/1712.05742v2] On the minimal ranks of matrix pencils and the existence of a best approximate block-term tensor decomposition - [http://arxiv.org/abs/2104.14187v1] On the multiplicity spaces for branching to a spherical subgroup of minimal rank - [http://arxiv.org/abs/1811.03813v1] The trouble with tensor ring decompositions - [http://arxiv.org/abs/2311.16912v2] Continuous optimization methods for the graph isomorphism problem - [http://arxiv.org/abs/1711.08885v2] On the Parallel Parameterized Complexity of the Graph Isomorphism Problem - [http://arxiv.org/abs/quant-ph/0309068v2] Quantum lower bounds for the set equality problems

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.add((i, j))
        return (n, edges)

    def compute_minimal_rank(graph):
        n, edges = graph
        adjacency_matrix = [[0] * n for _ in range(n)]
        for u, v in edges:
            adjacency_matrix[u][v] = 1
            adjacency_matrix[v][u] = 1
```

```

rank = 0
while True:
    found_zero_row = False
    for i in range(n):
        if all(adjacency_matrix[i][j] == 0 for j in range(n)):
            found_zero_row = True
            break
    if not found_zero_row:
        break

    rank += 1
    for i in range(n):
        if adjacency_matrix[i][i] != 0:
            for j in range(n):
                if adjacency_matrix[j][j] == 0 and any(adjacency_matrix[j][k] * adjacency_
                    adjacency_matrix[j][j] = 1
                break

    return rank

def is_isomorphic(graph1, graph2):
    n1, edges1 = graph1
    n2, edges2 = graph2
    if n1 != n2:
        return False

    for perm in itertools.permutations(range(n1)):
        permuted_edges = {(perm[u], perm[v]) for u, v in edges1}
        if permuted_edges == edges2:
            return True
    return False

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    instances_tested = 0
    total_rank_sum = 0

    for _ in range(5): # Ensure at least 30 instances per seed
        graph1 = generate_random_graph(n)
        graph2 = generate_random_graph(n)

        if is_isomorphic(graph1, graph2):
            rank = compute_minimal_rank(graph1)
            total_rank_sum += rank
            instances_tested += 1

    if instances_tested == 0:
        return {
            "metric_name": "minimal_rank",
            "metric_value": None,
            "instances_tested": instances_tested,
            "conjecture_holds": False,
            "counterexample": "no_isomorphic_instances"
        }

```

```

        mean_rank = total_rank_sum / instances_tested
        results.append(mean_rank)

correlation_coefficient = calculate_correlation(n_values, results)

return {
    "metric_name": "minimal_rank",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "conjecture_holds": correlation_coefficient >= 0.9,
    "counterexample": ""
}

def calculate_correlation(x, y):
    n = len(x)
    if n != len(y):
        raise ValueError("x and y must have the same length")

    mean_x = sum(x) / n
    mean_y = sum(y) / n

    numerator = sum((xi - mean_x) * (yi - mean_y) for xi, yi in zip(x, y))
    denominator = math.sqrt(sum((xi - mean_x) ** 2 for xi in x)) * math.sqrt(sum((yi - mean_y) ** 2 for yi in y))

    if denominator == 0:
        return None

    return numerator / denominator

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

        if "metric_value" not in trial_result or trial_result["metric_value"] is None:
            continue

        results.append(trial_result["metric_value"])

    mean_value = sum(results) / len(results)
    std_value = math.sqrt(sum((x - mean_value) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r >= 0.9) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(r < 0.9 for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result < 0.9)
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_below_0.9\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE support_fraction_below_threshold")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
c_instances'}
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': None, 'instances_tested': 0, 'conjecture_hold

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dfa91cee.py", line 136, in <module>
    mean_value = sum(results) / len(results)
                  ~~~~~^~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that it was unable to verify the conjecture's conditions. | next: Re-run the test with proper error handling and ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12672
2	preregistration	ollama_remote	glm4:latest	0	0	5957
3	novelty	ollama_remote	glm4:latest	0	0	4685
4	novelty	ollama_remote	glm4:latest	0	0	6596
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17450
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10255
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13641
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13032

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	8042

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 92328 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/f36368cc8b51.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f36368cc8b51.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f36368cc8b51.tar.gz (if generated)